

FastShip: Shipment Tracking API

A persistent, production-style backend built with FastAPI and PostgreSQL

THE PROBLEM

I needed a backend system to manage shipments, but a simple setup wouldn't hold up. The system required persistent storage, structured data, and the ability to handle real-world operations like creating, updating, and tracking shipments. Temporary in-memory storage wasn't reliable and couldn't scale.

WHAT I DID

I built a REST API using FastAPI, connected to a PostgreSQL database for persistent storage, with a shipment table covering ID, origin, destination, status, and timestamps. I chose PostgreSQL for its strength with structured data and its wide use in production systems, used SQLAlchemy to interact with the database, and implemented full CRUD operations for shipment management. For development and deployment, I set up PostgreSQL with Docker and connected it to the FastAPI application using environment variables stored in a `.env` file.

THE OUTCOME

The API creates, reads, updates, and deletes shipment records with real database persistence — data survives server restarts, and the system's structure closely mirrors a real-world backend service. The project gave me hands-on experience with backend architecture, database design, and integrating FastAPI with PostgreSQL in a practical setup.

× Generic AI Version

Developed a scalable shipment management system using modern backend technologies.

✓ My Version

Built a shipment management API using FastAPI and PostgreSQL to store and manage shipment data with real database persistence.

Backend developer focused on FastAPI, PostgreSQL, and building practical, secure APIs.

Contact me to build backend APIs or database-driven systems.